Seguridad básica: usuarios en memoria y HTTP Basic

2. HTTP Basic y usuarios en memoria

Objetivo de la sesión

Entender qué hace Spring Security antes de llegar al controller

FILTRAR

Interponer una primera barrera delante de Spring MVC

IDENTIFICAR

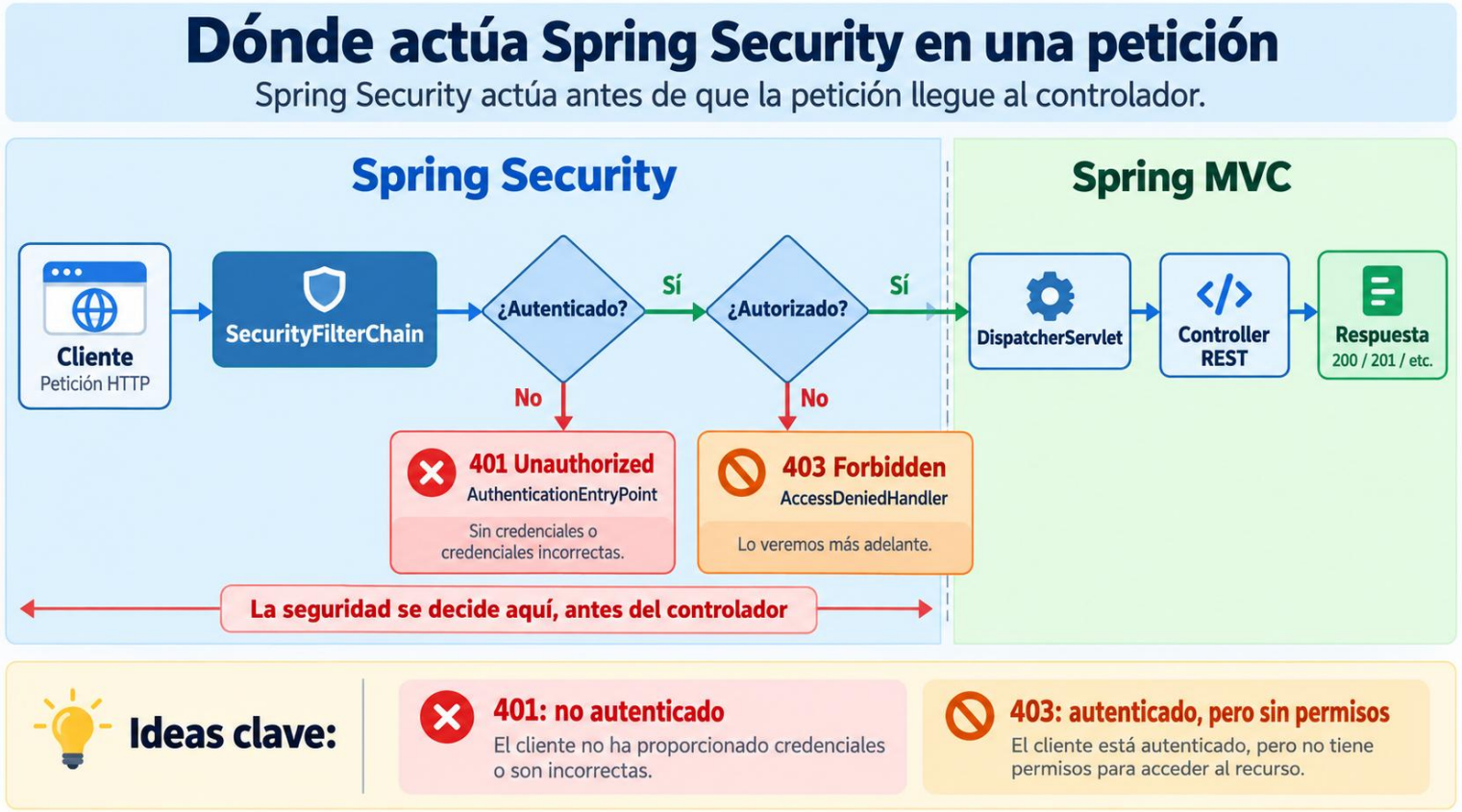
Separar autenticación y autorización

RESPONDER

Dar al 401 el mismo formato que al resto de la API

Estado intermedio deliberado: usuarios en memoria + HTTP Basic → PostgreSQL + BCrypt → JWT

Spring Security actúa antes de Spring MVC



Efecto inmediato

```
curl -i
http://localhost:8080/api/v1/libros
```

401 Unauthorized

Por defecto, Spring Security exige autenticación salvo que abras explícitamente una ruta.

Una petición puede ser rechazada sin que ningún controller llegue a ejecutarse.

Tres recorridos por la cadena de seguridad

2. Tres recorridos por la cadena de seguridad

Según las credenciales que envía el cliente, la petición puede seguir caminos distintos en Spring Security.
En los tres casos, Spring Security actúa antes de Spring MVC.



Autenticación ≠ autorización

3. Autenticación vs. autorización

La autenticación responde a la pregunta «¿Quién eres?». La autorización responde a «¿Puedes acceder a este recurso?».
Son pasos distintos pero relacionados: primero se identifica al usuario y después se comprueba si tiene permisos.

Analogía en la vida real

Acceso a un edificio con tarjeta

1 Autenticación

¿Quién eres?

TARJETA → El lector valida la tarjeta → **Identidad verificada**
Sabemos quién eres (por ejemplo, un empleado).

2 Autorización

¿Puedes abrir esta puerta?

TARJETA → Se comprueba si tienes permiso

- **Acceso permitido**
  Puedes entrar porque tienes permiso para esta zona.
- **Acceso denegado**
  Tu tarjeta es válida, pero no tienes permiso para esta puerta.

En una API con Spring Security

Primero se autentica el usuario y después se comprueban sus permisos.

1 Autenticación

¿Quién eres?

Cliente → `Authorization: Basic ...`
Usuario + contraseña (codificados en Base64)
→ **Spring Security**
AuthenticationManager valida las credenciales
→ **Usuario autenticado**
Se crea un Authentication en el SecurityContext.

2 Autorización

¿Puedes acceder a este recurso?

Cliente → `GET /api/v1/libros`
Petición a una ruta protegida
→ **Spring Security**
Comprueba los permisos (hasRole, hasAuthority, ...) según las reglas del `SecurityFilterChain`.

- **Acceso permitido**
  El usuario tiene permiso para este recurso.
- **Acceso denegado**
  El usuario está autenticado, pero no tiene permiso. (403 Forbidden)

HTTP Basic: codificar no es cifrar

4. HTTP Basic: codificar no es cifrar

HTTP Basic envía el usuario y la contraseña en cada petición, codificados en Base64.
Base64 es una transformación de texto: no oculta las credenciales, cualquiera puede decodificarlas.

1 Credenciales del usuario

El cliente prepara una cadena con usuario y contraseña separados por ":".

```
usuario:usuario123
```

2 Codificación en Base64

La cadena se codifica en Base64.

```
echo -n "usuario:usuario123" | base64
dXN1YXJpbzp1c3VhcmlvMTIz
```

3 Petición HTTP

El cliente incluye el resultado en la cabecera Authorization.



Cliente

```
GET /api/v1/libros HTTP/1.1
Host: ejemplo.com
Authorization: Basic dXN1YXJpbzp1c3VhcmlvMTIz
Accept: application/json
```

4 Decodificación en el servidor

El servidor decodifica la cabecera para obtener el usuario y la contraseña.



Servidor

```
echo -n "dXN1YXJpbzp1c3VhcmlvMTIz" | base64 -d
usuario:usuario123
```

Puntos clave

- Las credenciales se envían en cada petición.
- Se codifican en Base64, que es una transformación, **no un cifrado.**
- Spring Security las decodifica y valida el usuario y la contraseña.
- Es una forma sencilla de autenticación, útil para entornos de pruebas o desarrollo.

Importante

Base64 no oculta la información.
Cualquiera que intercepte la petición puede decodificar el usuario y la contraseña.

Solo debe usarse sobre HTTPS.
En HTTP las credenciales viajan en texto claro y pueden ser interceptadas fácilmente.

Ejemplo rápido de decodificación

Puedes comprobarlo tú mismo con base64 en Linux:

```
echo -n "usuario:usuario123" | base64          # codifica
echo -n "dXN1YXJpbzp1c3VhcmlvMTIz" | base64 -d  # decodifica
```

Usuarios en memoria: una solución provisional

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withUsername("user")
            .password("{noop}user123")
            .roles("USER")
            .build();

    UserDetails admin = User.withUsername("admin")
            .password("{noop}admin123")
            .roles("ADMIN")
            .build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

Qué debes entender

- InMemoryUserDetailsManager implementa UserDetailsService.
- {noop} significa contraseña en texto claro: solo para aprender el mecanismo.
- .roles("USER") crea internamente ROLE_USER.
- Los usuarios desaparecen al reiniciar: todavía no existe persistencia.

Siguiente sesión: PostgreSQL + BCrypt

Una primera política de rutas

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http)
        throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET,
"/api/v1/libros/**")
                .permitAll()
            .anyRequest()
                .authenticated()
        )
        .csrf(AbstractHttpConfigurer::disable)
        .httpBasic(Customizer.withDefaults())
        .build();
}
```

Léela de arriba abajo

`GET /libros/**` — **Público**

`cualquier otra` — **Usuario autenticado**

`httpBasic` — **Mecanismo de autenticación**

El orden importa: gana la primera regla que encaja.

Dos decisiones que conviene entender

1. El orden de las reglas

Spring Security evalúa authorizeHttpRequests de arriba abajo.

```
.requestMatchers(GET,
"/api/v1/libros/**").permitAll()
.anyRequest().authenticated()
```

Reglas concretas primero; reglas generales al final.

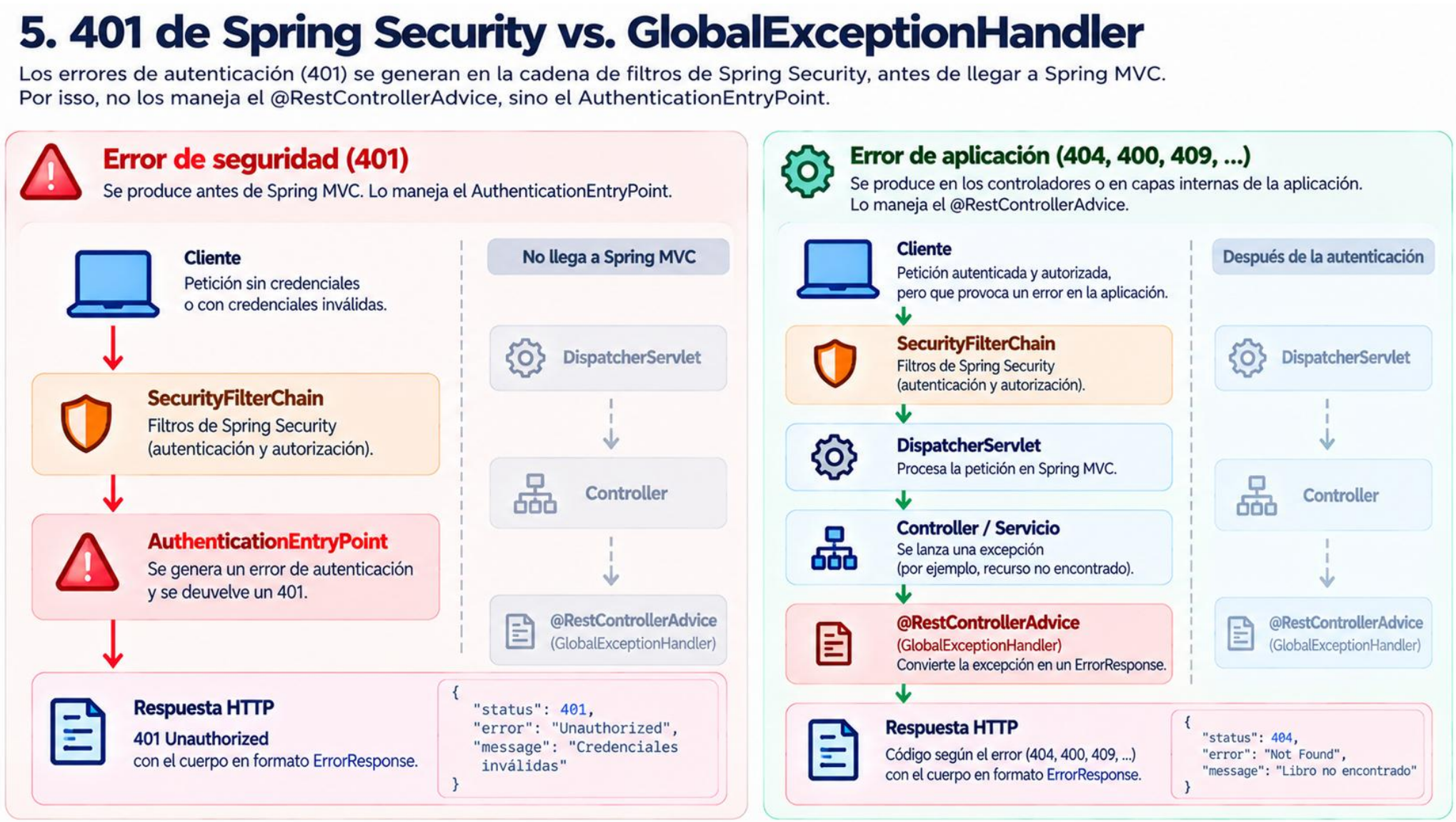
2. Por qué desactivamos CSRF aquí

En esta API las credenciales se envían explícitamente en Authorization y no mediante cookies de sesión.

```
.csrf(AbstractHttpConfigurer::di
sable)
```

No es una regla universal: depende de cómo se transporten las credenciales.

¿Por qué GlobalExceptionHandler no ve el 401?



AuthenticationEntryPoint: el 401 antes de Spring MVC

```
@Component
@RequiredArgsConstructor
public class ErrorResponseAuthenticationEntryPoint
        implements AuthenticationEntryPoint {

    private final JsonMapper jsonMapper;

    @Override
    public void commence(HttpServletRequest request,
                         HttpServletResponse response,
                         AuthenticationException ex) throws
IOException {
        ErrorResponse error = new ErrorResponse(...);
        response.setStatus(401);
        response.setContentType("application/json");
        response.getWriter().write(
                jsonMapper.writeValueAsString(error));
    }
}
```

No memorices la mecánica

- Existe porque el error ocurre antes del DispatcherServlet.
- No devuelve ResponseEntity: escribe directamente en HttpServletResponse.
- Reutiliza el mismo ErrorResponse que la API.
- JsonMapper convierte ese objeto a JSON.

Registrar el handler sin ligarlo a HTTP Basic

```
.exceptionHandling(exceptions ->
exceptions

.authenticationEntryPoint(customAuthentica
tionEntryPoint)
)
.csrf(AbstractHttpConfigurer::disable)
.httpBasic(Customizer.withDefaults())
```

¿Por qué exceptionHandling(...) separado?

Porque el formato del 401 no tiene por qué depender del mecanismo de autenticación. Cuando HTTP Basic desaparezca y llegue JWT, esta pieza seguirá teniendo sentido.

403 Forbidden

Usuario autenticado pero sin permisos

La pieza equivalente es AccessDeniedHandler

La implementaremos más adelante.

Un estado intermedio con dos problemas deliberados

1. Usuarios en el código

user y admin se reconstruyen en cada arranque.

Las contraseñas con {noop} están en texto claro.

No hay persistencia real.

2. Credenciales en cada petición

HTTP Basic vuelve a mandar usuario + contraseña continuamente.

Base64 no las oculta.

Necesitamos otro mecanismo.

Evolución del tema Memoria + Basic → PostgreSQL + BCrypt → JWT

Ideas clave

- Spring Security se sitúa antes de Spring MVC y protege las rutas por defecto.
- Autenticación responde a “¿quién eres?”; autorización a “¿puedes hacer esto?”.
- HTTP Basic envía usuario:contraseña en Base64 en cada petición: solo es aceptable sobre HTTPS.
- InMemoryUserDetailsManager y {noop} sirven para entender el mecanismo, no como solución final.
- Las reglas de authorizeHttpRequests se evalúan de arriba abajo.
- Los 401 de seguridad no pasan por GlobalExceptionHandler: los formatea AuthenticationEntryPoint.

Actividad 2.2: trasladar este estado intermedio a GameVault y comprobarlo con curl.